
Lab 02

Objective:

The objective of this lab is to learn about numpy, a python library, commonly used for implementing neural networks.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement vectors and matrixes.
- Perform vectors and matrix operations such as dot product, addition, subtraction, reshaping, etc
- Perform indexing and slicing.
- Implement random matrixes
- Implement algebraic operations such as transpose

Instructor Note:

As pre-lab activity, read and implement the tutorial:

<https://numpy.org/doc/stable/user/quickstart.html> .

4) Useful Concepts

1. **NumPy (Numerical Python):** It is a powerful Python library used extensively in numerical computing and data analysis. It provides support for large, multi-dimensional arrays and matrices, along with a collection of high-level mathematical functions to operate on these arrays efficiently.

Important Characteristics of NumPy:

- i. **Multi-dimensional Arrays:** NumPy's primary data structure is the ndarray, a multi-dimensional array object that can hold elements of the same type.
- ii. **Efficient Operations:** NumPy operations are implemented in C and Fortran, making them fast and efficient compared to Python's built-in data structures.
- iii. **Mathematical Functions:** NumPy provides a rich set of mathematical functions that operate element-wise on arrays, facilitating complex mathematical computations.
- iv. **Broadcasting:** NumPy supports broadcasting, which allows arithmetic operations on arrays of different shapes without explicitly converting them.
- v. **Integration with Other Libraries:** NumPy integrates well with other Python libraries for data analysis, such as pandas, matplotlib, and scikit-learn.

To use NumPy in your Python programs, you need to import it first. Here's how you can import NumPy:

```
import numpy as np
```

2. **Vector:** A vector, in the context of NumPy in Python, is a one-dimensional array that can store a sequence of numbers. It is a fundamental data structure used in various mathematical and scientific computations. In NumPy, you can create a vector using the *numpy.array* function:

```
#Load Library
import numpy as np

#Create a vector as a Row
vector_row = np.array([1,2,3])

#Create vector as a Column
vector_column = np.array([[1],[2],[3]])
```

-
3. **Matrix:** A matrix refers to a specific type of two-dimensional array. It is a fundamental data structure used for various mathematical operations, especially for implementing artificial neural networks. A matrix in NumPy is represented by the *numpy.ndarray* object with two dimensions.

It can be visualized as a rectangular grid of numbers arranged in rows and columns.

```
import numpy as np

# Creating a 2x3 matrix (2 rows, 3 columns)
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix)
```

Element-wise Operations: Matrices support element-wise arithmetic operations (addition, subtraction, multiplication, division) similar to vectors but applied to corresponding elements.

```
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])

# Element-wise addition
result = matrix1 + matrix2
print(result)
```

Matrix Multiplication: For matrix multiplication (dot product), you can use the `np.dot` function or the `@` operator in Python 3.5+:

```
result = np.dot(matrix1, matrix2)
# OR
result = matrix1 @ matrix2
print(result)
```

4. **Creating a Sparse Matrix:** Given data with very few non zero values you want to efficiently represent it. A frequent situation in Machine Learning is having a huge amount of Data; however most of the elements in the data are Zeros. For example, lets imagine a Matrix where the columns are all the products on Amazon and the rows signify whether a given user has bought that item before or not. Now as you might have guessed there would be many products which haven't been bought even a single time till now and thus a vast majority of elements would be Zero.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[0,0],[0,1],[3,0]])
print(matrix)

#Create Compressed Sparse Row(CSR) matrix
matrix_sparse = sparse.csr_matrix(matrix)
print(matrix_sparse)
```

5. **Indexing** in NumPy is the process of accessing and manipulating elements within arrays. It allows you to retrieve or modify specific elements, rows, columns, or sub-arrays in a NumPy array. Here are the key concepts:

- **Basic Indexing:**

1-D Arrays: You can access elements using their index positions, starting from 0:

```
import numpy as np

arr = np.array([10, 20, 30, 40])
print(arr[0]) # prints 10
print(arr[2]) # prints 30
```

2-D Arrays: Access elements using a pair of indices [row, column].

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix[0, 1]) # prints 2 (first row, second column)
print(matrix[1, 2]) # prints 6 (second row, third column)
```

- **Slicing:**

Allows you to access a range of elements.

1-D Arrays: arr[start:stop:step]

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[1:4]) # prints [20, 30, 40]
print(arr[:3]) # prints [10, 20, 30]
print(arr[::2]) # prints [10, 30, 50]
```

2-D Arrays: matrix[start_row:end_row, start_col:end_col]

```
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(matrix[0:2, 1:3]) # prints [[2, 3], [5, 6]]
```

- **Boolean Indexing:**

Use conditions to select elements.

```
arr = np.array([10, 20, 30, 40])
print(arr[arr > 20]) # prints [30, 40]
```

- **Fancy Indexing:**

```
arr = np.array([10, 20, 30, 40])
indices = [0, 2]
print(arr[indices]) # prints [10, 30]
```

6. Describing a Matrix

When you want to know about the shape size and dimensions of a Matrix.

```
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])

#View the Number of Rows and Columns
print(matrix.shape)

#View the number of elements (rows*columns)
print(matrix.size)

#View the number of Dimensions(2 in this case)
print(matrix.ndim)
```

7. **Applying operations to elements:** You want to apply some function to multiple elements in an array. Numpy's vectorize class converts a function into a function that can apply to multiple elements in an array or slice of an array.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Create a function that adds 100 to something
add_100 =lambda i: i+100

#Convert it into a vectorized function
vectorized_add_100= np.vectorize(add_100)

#Apply function to all elements in matrix
print(vectorized_add_100(matrix))
```

8. **Finding the max and min values:** We use Numpy's max and min functions:

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Return the max element
print(np.max(matrix))

#Return the min element
print(np.min(matrix))

#To find the max element in each column
print(np.max(matrix,axis=0))

#To find the max element in each row
print(np.max(matrix,axis=1))
```

9. **Calculating Average, Variance and Standard deviation:** When you want to calculate some descriptive statistics about an array.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Mean
print(np.mean(matrix))

#Standard Dev.
print(np.std(matrix))

#Variance
print(np.var(matrix))
```

10. Reshaping Arrays: When you want to reshape an array(changing the number of rows and columns) without changing the elements.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Reshape
print(matrix.reshape(9,1))

#Here -1 says as many columns as needed and 1 row
print(matrix.reshape(1,-1))

#If we provide only 1 value Reshape would return a 1-d array of that
length
print(matrix.reshape(9))

#We can also use the Flatten method to convert a matrix to 1-d array
print(matrix.flatten())
```

11. Transposing a vector or a Matrix: By transposing you interchange the rows and columns of a Matrix

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Transpose the matrix
print(matrix.T)
```

12. Finding the Determinant and Rank of a Matrix: The rank of a Matrix is the number of dimensions of the vector space spanned by its rows or columns.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Calculate the Determinant
print(np.linalg.det(matrix))

#Calculate the Rank
print(np.linalg.matrix_rank(matrix))
```

13. Getting the Diagonal of a Matrix: When you need to extract only the diagonal elements of a matrix

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Print the Principal diagonal
print(matrix.diagonal())

#Print the diagonal one above the Principal diagonal
print(matrix.diagonal(offset=1))

#Print the diagonal one below Principal diagonal
print(matrix.diagonal(offset=-1))
```

14. Calculating Dot Products

```

#Load Library
import numpy as np

#Create vector-1
vector_1 = np.array([ 1,2,3 ])

#Create vector-2
vector_1 = np.array([ 4,5,6 ])

#Calculate Dot Product
print(np.dot(vector_1,vector_2))

#Alternatively you can use @ to calculate dot products
print(vector_1 @ vector_2)

```

15. **Generating Random values:** Numpy offers a wide variety of means to generate Random Numbers. Moreover, It can sometimes be useful to return the same random numbers to get predictable, repeatable results. We can do so by setting the ‘Seed’ (An Integer) of the pseudorandom generator. Random processes with the same seed would always produce the same result.

```

#Load Library
import numpy as np

#Set seed
np.random.seed(1)

#Generate 3 random integers b/w 1 and 10
print(np.random.randint(0,11,3))

#Draw 3 numbers from a normal distribution with mean 1.0 and std 2.0
print(np.random.normal(1.0,2.0,3))

```

16. Solved Lab Activites

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
1	10	Low	CLO-2
2	10	Low	CLO-2
3	10	Low	CLO-2
4	10	Low	CLO-2
5	10	Low	CLO-2
6	10	Low	CLO-2
7	10	Low	CLO-2
8	10	Low	CLO-2
9	10	Low	CLO-2

Activity 1:

Numpy to illustrate 2-D Matrix.

Solution:

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6]])
print(matrix)
```

Activity 2:

Numpy to illustrate indexing and slicing.

```
#Load Library
import numpy as np

#Create a vector as a Row
vector_row = np.array([ 1,2,3,4,5,6 ])

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Select 3rd element of Vector
print(vector_row[2])

#Select 2nd row 2nd column
print(matrix[1,1])#Select all elements of a vector
print(vector_row[:])#Select everything up to and including the
3rd element
print(vector_row[:3])#Select the everything after the 3rd element
print(vector_row[3:])#Select the last element
print(vector_row[-1])#Select the first 2 rows and all the columns of
the matrix
print(matrix[:2,:])#Select all rows and the 2nd column of the
matrix
print(matrix[:,1:2])
```

Activity 3:

Describing the matrix.

```
import numpy as np
```

```
#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])#View the Number of
Rows and Columns
print(matrix.shape)#View the number of elements (rows*columns)
print(matrix.size)#View the number of Dimensions(2 in this case)
print(matrix.ndim)
```

Activity 4:

Finding mix and max values.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)#Return the max element
print(np.max(matrix))#Return the min element
print(np.min(matrix))#To find the max element in each column
print(np.max(matrix,axis=0))#To find the max element in each row
print(np.max(matrix,axis=1))
```

Output

Shown above

Activity 5:

Reshaping array.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)#Reshape
print(matrix.reshape(9,1))#Here -1 says as many columns as needed
and 1 row
print(matrix.reshape(1,-1))#If we provide only 1 value Reshape
would return a 1-d array of that lengthprint(marix.reshape(9))#We
can also use the Flatten method to convert a matrix to 1-d
arrayprint(matrix.flatten())
```

Output

Shown above

Activity 6:

python program to transpose an array.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)#Transpose the matrix
print(matrix.T)
```

Output

Shown above

Activity 7:

Finding dot product.

```
#Load Library
import numpy as np

#Create vector-1
vector_1 = np.array([ 1,2,3 ])#Create vector-2
vector_2 = np.array([ 4,5,6 ])#Calculate Dot Product
print(np.dot(vector_1,vector_2))#Alternatively you can use @ to
calculate dot products
print(vector_1 @ vector_2)
```

Output

Shown above

Activity 8:

Finding dot product.

```
#Load Library
import numpy as np

#Create vector-1
vector_1 = np.array([ 1,2,3 ])#Create vector-2
vector_2 = np.array([ 4,5,6 ])#Calculate Dot Product
print(np.dot(vector_1,vector_2))#Alternatively you can use @ to
calculate dot products
print(vector_1 @ vector_2)
```

Output

Shown above

Activity 9:

Generating random values.

```
#Load Library
import numpy as np#Set seed
np.random.seed(1)#Generate 3 random integers b/w 1 and 10
print(np.random.randint(0,11,3))#Draw 3 numbers from a normal
distribution with mean 1.0 and std 2.0
print(np.random.normal(1.0,2.0,3))
```

Output

Shown above

17. Graded Lab Tasks

The purpose of this lab task is to practice creating, accessing, and slicing matrices using NumPy in Python.

Lab Task 1

Instructions:

- Matrix Declaration:**
 - Create a 3x3 matrix of integers.
 - Create a 4x4 matrix of floats.
- Matrix Indexing:**
 - Access the element at the 2nd row and 3rd column of the integer matrix.
 - Change the element at the 1st row and 4th column of the float matrix to 20.5.
- Matrix Slicing:**
 - Extract the first two rows and columns of the integer matrix.
 - Extract the last row of the float matrix.

Submission:

- Save your code in a Python script named `matrix_lab.py`.
- Submit your script file by the due date.

Lab Task 2

The purpose of this lab task is to help students to practice creating matrices and performing various matrix operations using NumPy in Python.

Instructions:

- Matrix Declaration:**

Create the following matrices:

- A 3x3 integer matrix named `matrix_int`.
- A 3x3 float matrix named `matrix_float`.

2. Basic Matrix Operations:

- Perform element-wise addition of `matrix_int` and `matrix_float`.
- Perform element-wise multiplication of `matrix_int` and `matrix_float`.
- Find the transpose of `matrix_int`.

3. Advanced Matrix Operations:

- Perform matrix multiplication of `matrix_int` and `matrix_float`.
- Calculate the determinant of `matrix_int`.
- Find the inverse of `matrix_float`.

Submission:

- Save your code in a Python script named `matrix_operations_lab.py`.
- Submit your script file by the due date.

Lab Task 3

For this project, you will use a simple dataset of student grades. The dataset is provided below as a list of lists, where each sublist represents a student and contains their ID, math grade, science grade, and English grade.

```
data = [ [1, 85, 78, 92], [2, 76, 85, 88], [3, 90, 92, 95], [4, 68, 72, 70], [5, 88, 89, 94] ]
```

Instructions:

- **Setup:**
 - Import the necessary libraries: `numpy` and `matplotlib`.
- **Load and Display the Dataset:**
 - Convert the provided dataset into a NumPy array.
 - Display the dataset.
- **Extract and Analyze Specific Data Columns:**
 - Extract the math, science, and English grades as separate arrays.
 - Calculate and display the average grade for each subject.
- **Perform Basic Statistical Analysis:**

-
- Calculate the overall average grade for each student.
 - Find the student with the highest and lowest average grade.

- **Manipulate and Transform the Data:**

- Add 5 bonus points to each student's math grade.
- Calculate the new average grade for each student after adding the bonus points.

Submission:

- Save your code in a Python script named `data_analysis_lab.py`.
- Submit your script file by the due date.